SMART WATER MANAGEMENT FOR URBAN GREEN INFRASTRUCTURE

A Low-Cost, Network-Resilient Edge-IoT Irrigation Node with Deep-Learning Soil Analysis

FINAL PROJECT REPORT

Course: Design of Smart Cities

Table of Contents

Table of Contents 2
Abstract 4
1. Introduction 4
1.1 Purpose 4
1.2 Scope 4
1.3 Smart-city context 5
1.4 Definitions, acronyms and abbreviations 5
1.5 Overview of this document 5
2. Software Requirements Specification (SRS) 6
2.1 Product perspective 6
2.2 Product functions 6
2.3 User characteristics and stakeholders 6
2.4 Functional requirements 6
2.5 Non-functional requirements 7
2.6 External interface requirements 7
2.7 Constraints, assumptions and dependencies 8
3. Software Design Specification (SDS) 8
3.1 Architectural design 8
3.2 Deployment design 9
3.3 Hardware design and component selection 9
3.4 Circuit design 10
3.5 Data design 11
3.6 Component and module design 12
3.7 Control-law design 12
3.8 Machine-learning pipeline design 13
3.9 Design decisions and rationale 13
4. Implementation 13
4.1 Firmware 13
4.2 Calibration procedure 14
4.3 Host telemetry collector 14
4.4 Classifier pipeline 14
4.5 Dashboard and advisory 14

4.6 Prototype assembly 14
4.7 Technology stack 15
5. Testing 15
5.1 Test strategy 15
5.2 Functional and fault-injection testing 15
5.3 Control-law validation 16
5.4 Classifier evaluation 17
5.5 Telemetry reliability 18
6. Outcome 18
6.1 Requirements traceability 18
6.2 Quantitative outcomes 19
6.3 Unique outcome: measuring evaluation inflation in soil-image classification 19
6.4 Outcome against the smart-city objective 20
6.5 Limitations 20
6.6 Future scope 20
6.7 Conclusion 21
References 21

Abstract

Cities irrigate a great deal of land. Parks, urban forests, road medians, green belts, traffic islands, institutional lawns, green roofs and community gardens all need water, and in most municipalities that water is delivered on a fixed timer. A timer cannot tell that it rained yesterday, that one bed drains faster than another, or that a valve has failed. The result is simultaneous over-watering and under-watering across the same green space, in cities that are often already under water stress.

This project specifies, designs, implements and tests a low-cost irrigation node for exactly that setting. Built around an ESP32-WROOM-32, it measures soil moisture with a capacitive probe, measures air temperature and humidity with a DHT11, and switches a pump through a relay driven by a BC547B transistor. A two-threshold hysteresis rule — water below 30 % moisture, stop above 60 % — replaces the timer, and the complete decision loop executes on the microcontroller once per second.

The architectural decision that matters most in a smart-city context is where that decision is taken. The node does not ask a server whether to irrigate. A host computer logs its telemetry, classifies soil photographs and produces weather-aware watering advice, but is physically incapable of operating the relay. A city deploying hundreds of such nodes therefore gains a monitoring layer without acquiring a single point of failure.

The node was assembled for approximately ₹970 (about US$11–12) and validated against seven fault-injection tests and 8,171 logged telemetry records. Testing established both what the system does and what it cannot see: loss of pump power goes undetected while the firmware continues to report that irrigation is in progress. The project's unique outcome, set out in Section 6.3, is a measurement of how much conventional evaluation inflates soil-image classification accuracy — 45.0 to 47.9 percentage points on this dataset.

1. Introduction

1.1 Purpose

This document specifies the requirements, design, implementation, testing and outcome of an edge-based irrigation controller for urban green infrastructure. It is intended to be read by the project supervisor and evaluators, and to be sufficient for another engineer to rebuild and re-evaluate the system from the information it contains.

1.2 Scope

The system covers measurement of soil and ambient conditions, autonomous actuation of an irrigation pump, telemetry capture, soil classification from photographs, weather-aware watering advice and a monitoring interface. It does not cover multi-zone valve control, wireless backhaul, enclosure design or volumetric metering of water delivered; these are identified as future scope in Section 6.6.

1.3 Smart-city context

Water is one of the core utility domains of a smart city, alongside energy, mobility and waste. Within that domain, irrigation of public green space is unusual in two respects: it is a large and continuous consumer of treated or recycled water, and it is almost always controlled open-loop, by schedule rather than by measurement.

Green infrastructure is simultaneously becoming more important to urban planning, because vegetation moderates the urban heat-island effect, manages stormwater runoff and improves air quality. Cities are therefore being asked to maintain more green space, more reliably, without increasing water consumption. Demand-driven irrigation is the direct response to that pressure.

A second requirement comes from the nature of city-scale deployment. A system that works as a single demonstration unit may fail as a fleet of four hundred. Three properties decide whether it scales: unit cost, tolerance of intermittent connectivity, and whether a failed unit announces itself or fails silently. All three are treated as first-class requirements in Section 2 rather than as afterthoughts.

1.4 Definitions, acronyms and abbreviations

Term	Meaning
ADC	Analogue-to-digital converter; the ESP32's is 12-bit (0–4095)
Dead band	The moisture range between the two thresholds in which the pump holds its existing state
Edge computing	Processing and decision-making performed on the device itself rather than on a remote server
GPIO	General-purpose input/output pin
Hysteresis	Control using two thresholds rather than one, so that the switching point depends on the current state
Open-loop control	Control in which the result of an action is never measured or fed back
SDS	Software Design Specification
Source-disjoint evaluation	Evaluation in which every sample derived from a held-out photograph is excluded from training
SRS	Software Requirements Specification
SVM	Support vector machine
Telemetry	Measurement data transmitted from the node for monitoring

Table 1.1: Definitions and abbreviations.

1.5 Overview of this document

Section 2 is the Software Requirements Specification: what the system must do and what it must survive. Section 3 is the Software Design Specification: architecture, hardware, data and module design, with the rationale for each major decision. Section 4 describes the implementation, Section 5 the testing, and Section 6 the outcome, including a requirements traceability matrix and the project's unique outcome.

2. Software Requirements Specification (SRS)

2.1 Product perspective

The system is a self-contained irrigation node intended to be replicated across heterogeneous urban green space, reporting upward to a municipal monitoring layer. It replaces a timer-based irrigation controller. It is not a component of an existing product line and has no external system dependencies at run time other than an optional weather API used for advisory purposes only.

2.2 Product functions

- Measure soil moisture and ambient temperature and humidity.
- Decide autonomously when to irrigate, and actuate a pump accordingly.
- Report measurements and actuation state continuously.
- Classify soil type and condition from a photograph.
- Produce a watering advisory from soil state and a weather forecast.
- Present live and historical data through a monitoring interface.

2.3 User characteristics and stakeholders

Stakeholder	Characteristics	Need addressed
Municipal parks and horticulture department	Non-specialist operators; manage many dispersed sites	Reduce water consumption; know which sites are actually being watered
City water utility	Analytical; works from consumption data	Lower discretionary demand; per-site consumption data
Smart-city operations centre	Monitors distributed assets in real time	Live asset status that does not depend on a perfect network
Maintenance crews	Field staff, often without instruments	Be directed to sites needing attention; read node state on sight
Residents and community gardeners	Non-technical, cost-sensitive	Affordable automation for terrace gardens and community plots

Table 2.1: Stakeholders and their characteristics.

2.4 Functional requirements

ID	Requirement	Priority
FR-1	The system shall measure volumetric soil moisture and report it as a percentage in the range 0–100 %.	High
FR-2	The system shall measure ambient air temperature and relative humidity.	Medium
FR-3	The system shall switch an irrigation pump automatically based on measured soil moisture, without human intervention.	High
FR-4	The control law shall use two thresholds with a dead band, so that sensor noise near a single set point cannot cause repeated relay switching.	High
FR-5	The system shall transmit a structured telemetry record at a fixed interval of one second.	High
FR-6	The system shall indicate its operating state locally, visible to a maintenance crew without instrumentation.	Medium
FR-7	The system shall classify soil type and soil condition from a photograph.	Medium

ID	Requirement	Priority
FR-8	The system shall generate a watering advisory combining soil state with a 48-hour weather forecast.	Medium
FR-9	The system shall provide a monitoring interface showing live values, history and model output, with report export.	Medium
FR-10	The system shall permit a supervisory manual override of the pump.	Low

Table 2.2: Functional requirements.

2.5 Non-functional requirements

ID	Requirement	Rationale at city scale
NFR-1	The irrigation decision shall be taken on the node itself and shall not depend on any network or remote service.	A fleet cannot assume reliable connectivity; network loss must not stop irrigation
NFR-2	Unit cost shall be low enough for deployment in quantity.	Cost per node, not total system cost, determines feasible coverage
NFR-3	Failure of a secondary sensor shall not stop the primary irrigation function.	Partial degradation is preferable to total outage across a fleet
NFR-4	The monitoring layer shall be incapable of actuating the pump.	Removes the remote attack surface and the single point of failure
NFR-5	The system shall be assembled from commodity components available from ordinary retailers.	Procurement, repair and replacement at municipal scale
NFR-6	Undetectable failure modes shall be identified and documented.	A dashboard that misreports asset state erodes operator trust

Table 2.3: Non-functional requirements.

2.6 External interface requirements

2.6.1 Hardware interfaces

The node interfaces with a DHT11 over a single-wire digital protocol on GPIO 4, with a capacitive moisture probe through a 12-bit analogue input on GPIO 34 (ADC1_CH6), and with a 5 V relay module through a BC547B transistor driven from GPIO 26. Two LEDs on GPIO 13 and GPIO 14 provide local state indication. Detailed pin assignments appear in Section 3.4.

2.6.2 Software interfaces

The node emits newline-delimited JSON over USB serial at 115200 baud. The host collector consumes that stream and writes a CSV telemetry log. The advisory module consumes a JSON forecast from the Open-Meteo HTTP API. Schemas for both are given in Section 3.5.

2.6.3 User interfaces

Two interfaces exist. On the node, a red LED indicates that the system is powered and running and a green LED blinks at 250 ms while the pump is active — readable by field staff without any instrument. On the host, a browser dashboard presents live values, history, classification results, the watering advisory and PDF export.

2.6.4 Communication interfaces

The node-to-host link is wired USB serial. No wireless interface is implemented in this phase. The advisory module requires outbound HTTPS to the forecast API; if that request fails, the advisory is omitted and all other functions, including irrigation, continue unaffected.

2.7 Constraints, assumptions and dependencies

- GPIO 34 is input-only and not 5 V tolerant, which constrains the probe to 3.3 V operation.
- The probe calibration constants are valid only at the supply voltage at which they were measured.
- The prototype is validated in a controlled soil container, not a live municipal installation.
- One probe represents the moisture of its irrigation zone; spatial variation within a zone is not captured.
- Mains power and a water source are assumed available at the site, as they generally are at managed urban green space.
- Volumetric water saving is not measured, because the prototype has no flow meter. Claims are therefore restricted to what was measured: actuation behaviour and control-law performance.

3. Software Design Specification (SDS)

3.1 Architectural design

The system is designed in four layers. The boundary between the first layer and the rest is deliberate and is the central design decision of the project: everything required to keep vegetation watered lives on the microcontroller, and every layer above it is additive.

- Sensing and control layer — the ESP32 reads both sensors, applies the control law, drives the relay and status LEDs, and emits one JSON record per second.
- Communication layer — a serial link at 115200 baud carries that stream to the host.
- Processing layer — a collector parses and stores the telemetry; the soil classifiers and advisory logic run here.
- Presentation layer — a browser dashboard provides live values, history, model output, watering advice and report export.

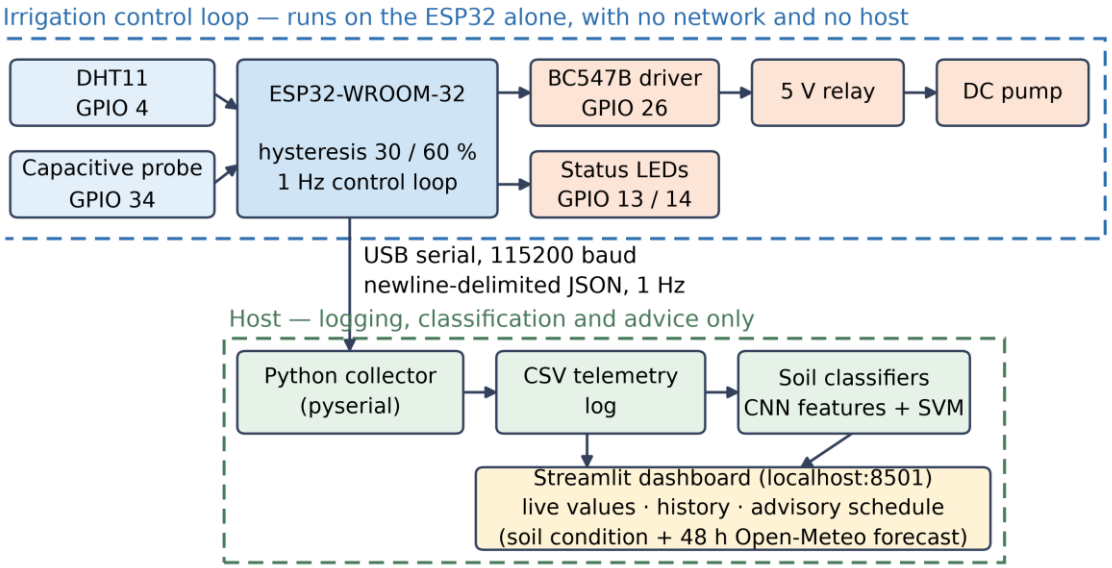


Figure 3.1: Layered system architecture. The control loop closes on the ESP32; everything beyond the serial link is monitoring and advisory and cannot actuate the relay.

3.2 Deployment design

At city scale the same node is replicated across heterogeneous green spaces. Each instance closes its own loop locally and reports upward. The municipal dashboard aggregates fleet status, consumption trends and maintenance alerts, but the direction of authority is strictly one-way: telemetry flows up, commands do not flow down. This satisfies NFR-1 and NFR-4 simultaneously, and changes the failure characteristics of the fleet — a network outage degrades visibility, not service.

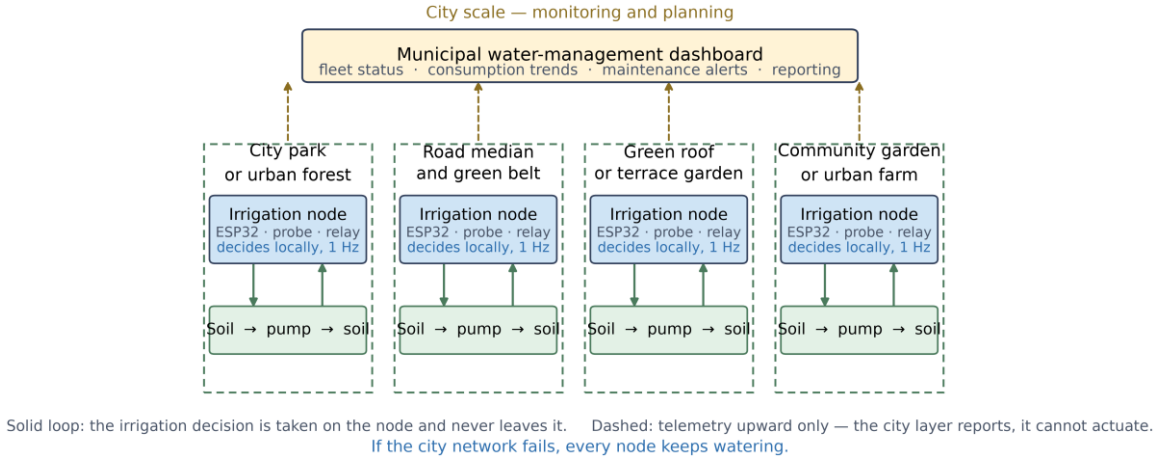


Figure 3.2: Urban deployment design. Each site closes its irrigation loop locally; the city layer monitors and plans but cannot actuate.

3.3 Hardware design and component selection

Component	Specification	Role
ESP32-WROOM-32 development board	Dual-core 32-bit MCU, 240 MHz, 12-bit ADC (0–4095), 3.3 V logic	Executes the control loop, drives the relay and LEDs, emits telemetry

Component	Specification	Role
Capacitive soil moisture sensor v1.2	Analogue output; capacitive sensing avoids the exposed electrodes that corrode in resistive probes; operated at 3.3 V	Primary measurement driving the irrigation decision
DHT11 temperature and humidity module	0–50 °C (±2 °C), 20–90 % RH (±5 %), single-wire digital	Ambient conditions for logging and evaporation-aware advice
5 V single-channel relay module	SPDT contacts, opto-isolated input, 5 V coil	Isolates control electronics from the pump circuit
BC547B NPN transistor with 1 kΩ base resistor	TO-92 general-purpose NPN	Relay driver; the GPIO cannot source the relay coil current
DC mini submersible pump	3–6 V DC	Delivers water when the relay closes
Status LEDs with 220 Ω resistors	Standard 5 mm LEDs	Local state indication for maintenance crews (FR-6)
5 V external supply	USB power bank or mains adaptor	Powers relay coil and pump on a separate rail with common ground

Table 3.1: Hardware components and their roles.

Unit cost is a design parameter, not an outcome (NFR-2). Four purchased modules account for ₹470; the balance covers the pump, transistor, LEDs, resistors, breadboard, wiring and tubing.

Item	Qty	Price (₹)
ESP32 development board (ESP-WROOM-32)	1	284
Capacitive soil moisture sensor v1.2	1	83
DHT11 temperature and humidity module	1	61
5 V single-channel relay module	1	42
Subtotal, itemised modules		470
Pump, BC547B, LEDs, resistors, breadboard, wiring, tubing	—	500
Total per node		970

Table 3.2: Bill of materials per node.

At ₹970 per node, instrumenting 100 irrigation zones costs approximately ₹97,000 in hardware and 400 zones approximately ₹3.9 lakh. These figures are arithmetic on the unit cost and exclude installation, enclosures, plumbing and maintenance; they indicate the order of magnitude at which this becomes a procurement question rather than a research one.

3.4 Circuit design

Both sensors are powered from the ESP32's 3.3 V rail. The DHT11 data line is pulled up to 3.3 V through 10 kΩ, and the probe's analogue output is read on GPIO 34. The relay is driven indirectly: GPIO 26 feeds the BC547B base through 1 kΩ, and the transistor switches the relay module input. The pump runs from a separate 5 V supply sharing a ground with the ESP32, so the relay contact switches only the pump circuit.

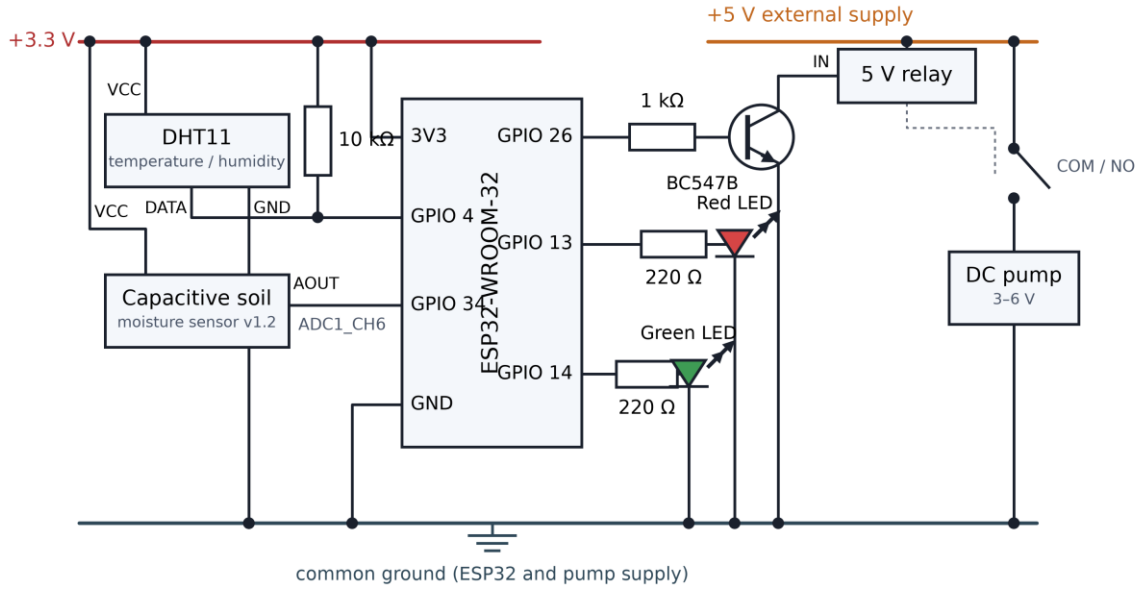


Figure 3.3: Circuit schematic showing the sensors on the 3.3 V rail, the BC547B relay driver, the status LEDs and the separately supplied pump.

ESP32 pin	Connected to	Through	Function
GPIO 4	DHT11 data line	10 kΩ pull-up to 3.3 V	Air temperature and humidity
GPIO 34	Moisture sensor analogue output	Direct (ADC1_CH6)	Soil moisture
GPIO 26	BC547B base	1 kΩ	Relay drive
GPIO 13	Red LED anode	220 Ω	System alive indicator
GPIO 14	Green LED anode	220 Ω	Pump running indicator
3.3 V	DHT11 and moisture sensor supply	—	Sensor power
GND	Common ground rail	—	Shared with the pump supply

Table 3.3: Pin assignments.

3.5 Data design

Three data structures carry information through the system.

Telemetry record emitted by the node, once per second:

Field	Type	Range / units	Notes
temperature	float or null	0–50 °C	null if the DHT11 read fails
humidity	float or null	20–90 % RH	null if the DHT11 read fails
moisture	integer	0–100 %	after mapping and clamping
soil_raw	integer	0–4095	raw 12-bit ADC value
pump	string	“ON” or “OFF”	commanded state, not measured

Table 3.4: Telemetry record schema.

Persisted telemetry log (CSV), one row per received record:

Column	Meaning
timestamp	Host-side receive time

Column	Meaning
soil_temp, soil_humidity, soil_moisture	Values from the node
status, action	Derived irrigation state and action taken
outside_temp, wind_speed, city	Context from the weather API

Table 3.5: Telemetry log schema.

Feature vector used by the soil-type classifier:

Component	Dimensions	Description
CNN embedding	1,280	Frozen ImageNet backbone, global average pooling
Colour histogram	96	32 bins per RGB channel, each normalised to sum to 1
Channel statistics	6	Mean and standard deviation per channel, divided by 255
Total	1,382	Input to the support vector machine

Table 3.6: Soil-type feature vector design.

3.6 Component and module design

Module	Responsibility	Key interface
smart_agri.ino	Sensing, control law, actuation, telemetry emission	USB serial, newline-delimited JSON
edge/serial_collector.py	Parse the stream, persist to CSV and status file	Reads serial; writes data/latest.json and sensor_log.csv
app/soil_predictor.py	Soil type inference	Image path or object in; class and probabilities out
app/soil_condition_predictor.py	Soil condition inference	Image in; condition, confidence and advice out
app/irrigation_scheduler.py	Forecast retrieval and watering advisory	Open-Meteo HTTP API; returns schedule windows
planner/crop_planner.py	Crop suitability reasoning and reporting	Soil and weather in; suitability report out
app/dashboard.py	Monitoring interface and orchestration	Streamlit on localhost:8501
training/source_disjoint_eval.py	Dual-protocol classifier evaluation	Writes results JSON consumed by the figure scripts

Table 3.7: Module responsibilities.

3.7 Control-law design

The probe is calibrated at two reference points measured on the assembled hardware: the raw ADC reading in dry air, A = 2590, maps to 0 % moisture, and the reading in water, W = 1500, maps to 100 %. Intermediate readings are interpolated and clamped:

$$M = clamp(\ 100 \times (r - A) / (W - A),\ 0,\ 100\ )$$

The clamp is a safety feature as much as a convenience: when the probe was deliberately disconnected during testing, the floating input read full scale and the clamp returned 0 % rather than a large negative value.

The actuation rule uses two thresholds rather than one. A single set point makes the relay chatter whenever readings hover near it, because real measurements fluctuate by a point or two. Here the pump turns on below 30 % moisture, turns off above 60 %, and between those

values holds its existing state. The 30-point dead band is wider than the observed sensor noise, so noise alone cannot toggle the relay. For a municipal fleet this directly extends relay and pump service life.

3.8 Machine-learning pipeline design

Soil images are resized to 224 × 224 pixels and passed through a convolutional neural network pre-trained on ImageNet with its weights frozen. Training a network from scratch would require far more images than this project can collect, so the network is used as a feature extractor and a support vector machine is trained on its output. Classifier hyperparameters were fixed before any evaluation and never tuned, and the feature scaler is fitted inside each training fold rather than on the full dataset, so that neither can leak information from the test data.

3.9 Design decisions and rationale

Decision	Alternative rejected	Reason
Control loop on the microcontroller	Cloud or gateway decision-making	A fleet cannot assume connectivity; network loss must not stop irrigation (NFR-1)
Two-threshold hysteresis	Single set point	Prevents relay chatter from sensor noise; extends actuator life (FR-4)
Monitoring layer has no actuation path	Bidirectional cloud control	Eliminates a remote failure and attack surface (NFR-4)
Capacitive probe	Resistive probe	No exposed electrodes, so far less corrosion in continuously moist soil
Transistor-driven relay module	Direct GPIO drive	The GPIO cannot source the relay coil current
Transfer learning with a frozen backbone	Training a CNN from scratch	Appropriate to a small image dataset
Separate pump supply with common ground	Single shared supply	Keeps motor switching transients off the logic rail

Table 3.8: Design decisions and their rationale.

4. Implementation

4.1 Firmware

The firmware is written in C/C++ for the Arduino core. After initialisation — configuring GPIO directions, starting the DHT11 driver, forcing the relay off, turning the red LED on and allowing the sensor two seconds to stabilise — it repeats the following cycle once per second:

- Read the DHT11 on GPIO 4. If the read fails, temperature and humidity are transmitted as null and the cycle continues; irrigation is never blocked by a failed air sensor (NFR-3).
- Read the capacitive probe on GPIO 34 as a raw 12-bit ADC value.
- Convert that value to a moisture percentage and clamp it to 0–100 %.
- Apply the hysteresis rule to determine the pump state.
- Update the LEDs — red steady for alive, green blinking at 250 ms while the pump runs.
- Emit one JSON record and delay one second.

4.2 Calibration procedure

Calibration is performed once per probe and is the step most likely to be done wrong in a field deployment, so it is documented explicitly. The probe is held in dry air and the raw ADC value recorded; it is then immersed in water to its marked depth and the value recorded again. These two constants are written into the firmware. Because both are measured at the probe's operating voltage, the probe must be powered at 3.3 V in service, as it was during calibration.

4.3 Host telemetry collector

A Python collector built on pyserial opens the serial port, parses each JSON line, writes the most recent reading to a status file and appends every record to a CSV log. The log is the basis for the control-law analysis in Section 5.3 and would, in a municipal deployment, be the feed into the city data platform.

4.4 Classifier pipeline

The training pipeline loads the source photographs, expands each into many samples through cropping, rotation and augmentation, extracts features with the frozen backbone, concatenates the colour descriptor where applicable, standardises the features and fits the support vector machine.

4.5 Dashboard and advisory

The dashboard is a Streamlit application served locally. It presents live sensor values as gauges, plots the logged history, accepts a soil photograph for classification, and combines the classified soil condition with a 48-hour Open-Meteo forecast to produce a watering advisory: whether irrigation is needed now, expected rainfall over the next 6, 24 and 48 hours, and which upcoming hours are best for watering given temperature and humidity. A suitability report can be exported as PDF. Supervisory manual pump control is provided (FR-10) as a dashboard function operating within the firmware's own state machine, not as a remote actuation path.

4.6 Prototype assembly

The node was assembled on a breadboard with the probe inserted into a sealed soil container that could be wetted and allowed to dry, giving repeatable control over the variable under test.

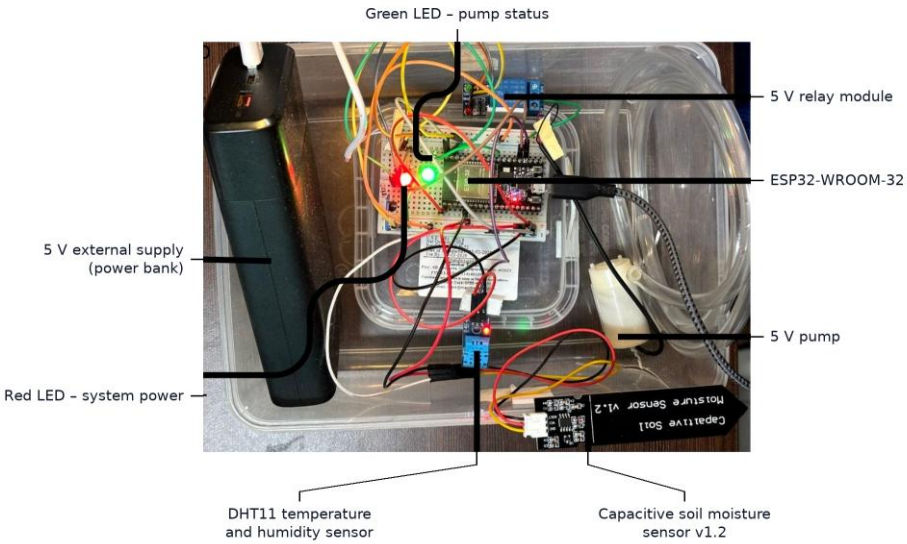


Figure 4.1: The assembled prototype node, with the ESP32, relay module, sensors, pump and external supply labelled.

4.7 Technology stack

Layer	Technology
Firmware	C/C++ on the Arduino core; DHT sensor library
Telemetry transport	USB serial, 115200 baud, newline-delimited JSON
Collector	Python 3.11, pyserial
Machine learning	TensorFlow / Keras (frozen backbones), scikit-learn
Data handling	NumPy, pandas
Interface	Streamlit, Plotly
Imaging	Pillow (PIL)
Reporting	ReportLab
External service	Open-Meteo forecast API

Table 4.1: Implementation technology stack.

5. Testing

5.1 Test strategy

Testing was organised around three questions. Does the node behave as specified under normal conditions and at the threshold boundaries? What happens when a component fails? And does the soil classifier perform as well on soil it has never seen as its headline accuracy suggests? The second and third questions matter most for a municipal deployment, because both concern whether the system can be trusted when nobody is watching it.

5.2 Functional and fault-injection testing

Test	Condition applied	Expected	Observed
TC-01	Moisture 65 %, 28 °C, 60 % RH	Pump off; red LED on; telemetry sent	As expected — pass

Test	Condition applied	Expected	Observed
TC-02	Moisture 22 %	Pump on; green LED blinking at 250 ms	As expected — pass
TC-03	Moisture 45 % with pump already on	Pump stays on	State held, no chatter — pass
TC-04	Moisture exactly 30 %	Pump off (rule is M < 30)	As expected — pass
TC-05	DHT11 data line removed during operation	Nulls in telemetry; irrigation continues	As expected — pass
TC-06	Moisture probe unplugged	Reading clamps to 0 %; pump turns on	Pump turned on as a fail-safe, but the fault itself was not detected
TC-07	Pump supply removed while running	Pump stops; ESP32 keeps running	Pump stopped but the firmware kept reporting it on — fail, undetected

Table 5.1: Functional and fault-injection test results.

Five of seven tests passed as specified. TC-06 is a fail-safe rather than a detection: the node waters because the reading falls to 0 %, but it does not know the probe is missing, and a probe that had drifted instead of being unplugged would produce no symptom at all.

TC-07 is the most consequential result. When the pump supply was cut, the pump stopped, but the firmware held its internal state at “on”, kept the green LED blinking and kept reporting irrigation. Nothing in the system could tell that no water was flowing. In a municipal deployment this failure would persist until vegetation visibly declined, and every record logged in the interval would state that the site was being watered. The cause is structural: the relay output is commanded but never measured.

5.3 Control-law validation

The control law was validated against 8,171 telemetry records captured over 33 logging sessions totalling about 4.5 hours, with a median sampling interval of 1.00 s. For each session the pump state was reconstructed from the recorded moisture under two rules: a single set point at 30 %, and the implemented hysteresis rule.

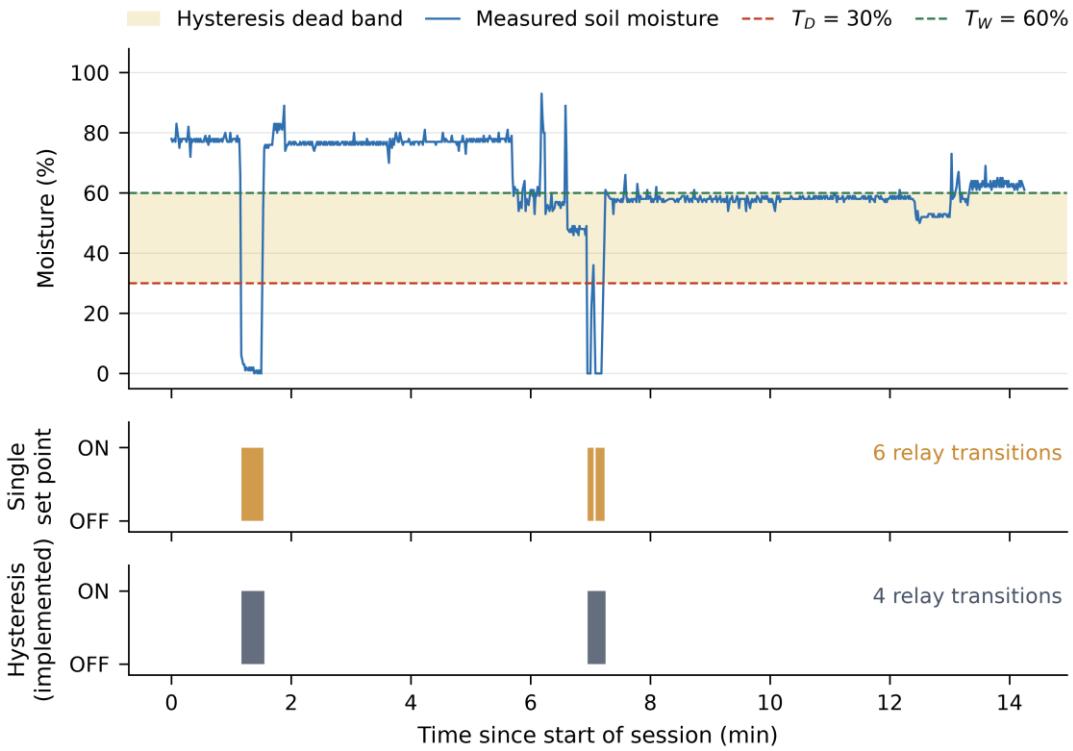


Figure 5.1: A measured 14-minute drying and rewetting cycle, with the pump state reconstructed under both rules. The single set point would have switched the relay six times; the hysteresis rule switched it four times.

Across all sessions the hysteresis rule reduced relay transitions by about 17 % relative to a single set point. The benefit is moderate because only 6.6 % of logged readings fall inside the 30–60 % dead band. In a continuously operating municipal installation, however, a 17 % reduction in actuation count accumulates directly into relay and pump service life.

5.4 Classifier evaluation

The image datasets were built by expanding a small number of source photographs into many samples through cropping, rotation and augmentation. This is standard practice with small datasets, but it introduces a hazard: if the expanded samples are shuffled before splitting into training and test sets, near-identical crops of the same photograph land on both sides of the split, and the classifier is partly tested on images it has effectively already seen.

The same classifier and features were therefore evaluated twice:

- Augmentation-level evaluation — the conventional approach, shuffling all expanded samples before splitting.
- Source-disjoint evaluation — every sample derived from a held-out photograph is excluded from training, so the classifier is tested only on soil it has genuinely never seen.

Task	Augmentation-level	Source-disjoint	Gap
Soil type (5 classes)	91.9–94.0 %	44.6–46.8 %	45.0–47.9 points
Soil condition (4 classes)	98.6–99.7 %	75.5–86.1 %	13.6–23.1 points

Table 5.2: Accuracy under the two evaluation protocols.

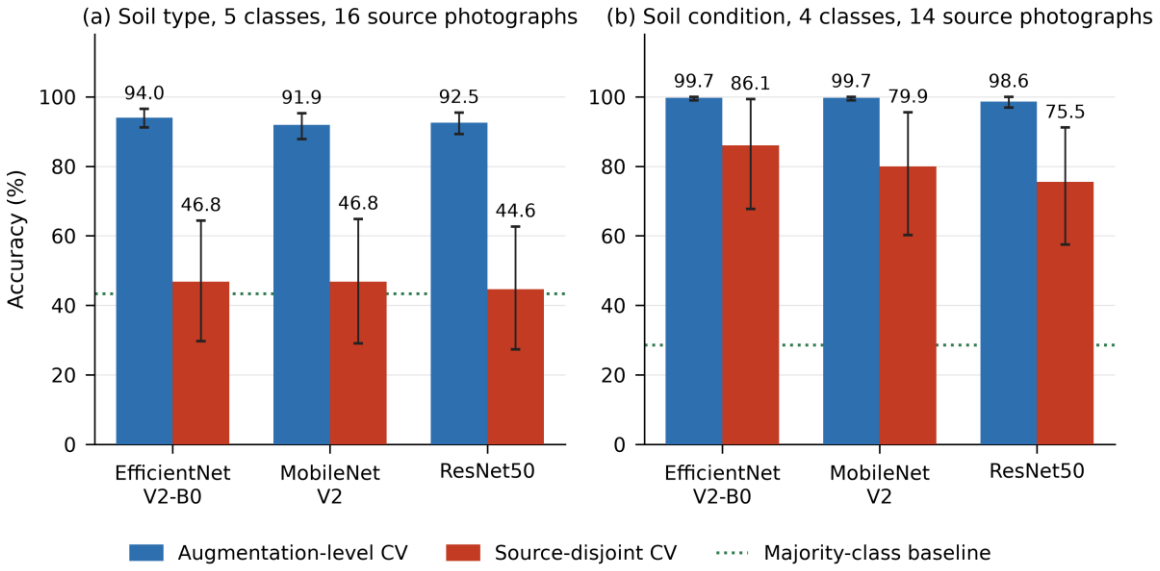


Figure 5.2: Accuracy under both protocols for three network backbones. The dotted line is the majority-class baseline.

Soil-type accuracy falls to 44.6–46.8 %, close to the 43.3 % obtainable by always guessing the most common class. The result held across three network architectures and five independent randomisations, so it is a property of the dataset rather than of one model. Soil condition holds up considerably better at 75.5–86.1 %, about three times its baseline, and is the more usable of the two classifiers.

5.5 Telemetry reliability

Across 7,949 within-session intervals the median time between records was 1.00 s, with an interquartile range of 1.00–2.00 s and a 95th percentile of 3.00 s, bounding how quickly the monitoring layer observes a change. Each fault in Table 5.1 was injected once, so the tests establish behaviour rather than its statistical consistency; repeated injection with instrumented timing is identified as future work.

6. Outcome

6.1 Requirements traceability

ID	Status	Evidence
FR-1	Met	Moisture reported 0–100 % from a two-point calibration; 8,171 records logged
FR-2	Met	DHT11 temperature and humidity logged throughout
FR-3	Met	TC-01 to TC-04: pump switched automatically from the moisture reading
FR-4	Met	TC-03 and Section 5.3: no chatter in the dead band; 17 % fewer transitions than a single set point
FR-5	Met	One JSON record per second; median interval 1.00 s
FR-6	Met	Red and green LEDs; verified in TC-01 and TC-02
FR-7	Partially met	Soil condition usable at 75.5–86.1 %; soil type not usable at 44.6–46.8 % (Section 5.4)
FR-8	Met	Advisory generated from soil condition and a 48 h Open-Meteo forecast
FR-9	Met	Streamlit dashboard with live values, history, model output and PDF export

ID	Status	Evidence
FR-10	Met	Supervisory manual override implemented in the dashboard
NFR-1	Met	Control loop executes entirely on the ESP32; verified with the host disconnected
NFR-2	Met	₹970 per node, about a quarter of the cloud-dependent baseline
NFR-3	Met	TC-05: irrigation continued on moisture alone after DHT11 failure
NFR-4	Met	By construction — no actuation path exists from the host to the relay
NFR-5	Met	All components sourced from a general electronics retailer
NFR-6	Met	TC-06 and TC-07 documented as undetectable with the present hardware

Table 6.1: Requirements traceability matrix.

6.2 Quantitative outcomes

Measure	Result
Cost per node	₹970 (approx. US$11–12)
Cost relative to cloud-dependent baseline	About one quarter
Control-loop execution	Entirely on-node, 1 Hz, no network dependence
Reduction in relay transitions vs single set point	About 17 %
Telemetry captured	8,171 records, 33 sessions, approx. 4.5 hours
Median telemetry interval	1.00 s (IQR 1.00–2.00 s)
Functional tests passed	5 of 7; 2 documented as undetected faults
Soil condition accuracy (unseen photographs)	75.5–86.1 %
Soil type accuracy (unseen photographs)	44.6–46.8 % — not deployable
Requirements met	15 of 16 fully; 1 partially

Table 6.2: Summary of measured outcomes.

6.3 Unique outcome: measuring evaluation inflation in soil-image classification

Beyond the working node, this project produced a result that is transferable to other smart-city machine-learning deployments and that, to our knowledge, has not previously been quantified for soil photographs.

Soil-image datasets are commonly built by expanding a handful of source photographs into thousands of samples through cropping and augmentation. Published accuracies for this task routinely exceed 90 %. By evaluating the same classifier, on the same features, under both the conventional protocol and a photograph-level split, this project measured how much of that accuracy is real:

- Soil type fell from 91.9–94.0 % to 44.6–46.8 % — a gap of 45.0 to 47.9 percentage points, and barely above the 43.3 % majority-class baseline.
- Soil condition fell from 98.6–99.7 % to 75.5–86.1 %.
- The soil-type gap held across three network architectures and five independent randomisations, confirming it as a property of the data rather than of one model.
- The same features identify which photograph a tile came from with 91.5 % accuracy, against a 6.3 % chance rate — direct evidence that the classifier was recognising photographs, not soils.
- Classes represented by a single source photograph score an F1 of exactly zero once that photograph is held out, because the class is then absent from training altogether.

The practical significance for smart-city design is this. Municipal systems increasingly justify operational and procurement decisions by citing model accuracy. An accuracy figure quoted without stating how the data were split, and how many genuinely independent samples underlie it, cannot support such a decision. On this dataset the difference between the two reporting conventions was large enough to invert the conclusion: a classifier that appears deployable at 94 % is, measured honestly, no better than guessing the most common class.

This finding is the basis of the research paper submitted alongside this report as the project outcome.

6.4 Outcome against the smart-city objective

The project set out to show that demand-driven irrigation of urban green space can be automated at a unit cost and a reliability profile appropriate to city-scale deployment. On the infrastructure side that objective was met: the node waters on measurement rather than on a timer, costs under a thousand rupees, continues to operate with the host and network both absent, degrades gracefully when its secondary sensor fails, and actuates its relay measurably less often than a naive controller.

The outcome also demonstrates a governance property that matters more at fleet scale than at prototype scale. Because the monitoring layer has no actuation path, adding city-wide visibility does not add a city-wide failure mode. An operations centre can lose its network and lose sight of four hundred nodes without any of them stopping work.

6.5 Limitations

- Undetected actuator failure. A pump that loses power is reported as running (TC-07).
- No probe validation. A drifted, fouled or dislodged probe continues to return plausible values, and the controller trusts them (TC-06).
- Volumetric water saving not measured. The prototype has no flow meter, so the report claims only what was measured and makes no claim about litres saved.
- Soil-type classification is not deployable at its measured accuracy on unseen photographs.
- Container validation only, over hours rather than a season.
- Single-point sensing: one probe represents a whole irrigation zone.
- Wired telemetry; wireless backhaul was not implemented or tested.

6.6 Future scope

- Add a current sensor on the pump supply, such as an ACS712 at about ₹145, which closes the TC-07 gap directly by showing whether the pump draws current. At roughly 15 % of unit cost this is the single highest-value addition.
- Implement a pulse-response probe check: deliver a known volume and verify that the moisture reading rises, catching a fouled or dislodged probe while correctly passing one that is merely miscalibrated.
- Install an inline flow meter so that water saving can be quantified in litres and reported per site.

- Migrate the telemetry link to Wi-Fi, LoRaWAN or NB-IoT for city-scale backhaul, keeping the control loop local.
- Expand to multi-zone valve control so one node can serve several beds.
- Collect several hundred field photographs across soil classes to make soil-type classification viable.
- Integrate with a municipal data platform and asset-management workflow so that alerts generate maintenance tickets automatically.
- Add enclosure design, surge protection and tamper resistance for street-level deployment.

6.7 Conclusion

Urban green infrastructure is watered largely by timers, in cities that increasingly cannot afford to waste water. This project specified, built and tested a node that waters by measurement instead, for about ₹970, with its decision loop held locally so that network failure degrades visibility rather than service. Seven fault tests established what it does and does not handle, and the one failure it cannot see — loss of pump power — is documented together with a ₹145 remedy.

The wider lesson concerns how smart-city systems are evaluated. A controller that reports irrigation it is not performing and a classifier that reports accuracy it does not have fail in the same way: both look correct from the outside. Neither failure was discovered by observing the system work; both were found only by testing specifically for them. For infrastructure a city intends to trust without supervision, that kind of adversarial testing is not an optional extra but the part of the work that determines whether the rest can be believed.

References

[1] A. Morchid, H. Qjidaa, R. El Alami, S. Mobayen, P. Skruch and B. Bossoufi, “Smart irrigation-based internet of things and cloud computing technologies for sustainable farming,” Scientific Reports, vol. 16, art. 5293, 2026.

[2] A. A. Abdelmoneim, H. N. Kimaita, C. M. Al Kalaany, B. Derardja, G. Dragonetti and R. Khadra, “IoT sensing for advanced irrigation management: a systematic review of trends, challenges, and future prospects,” Sensors, vol. 25, no. 7, art. 2291, 2025.

[3] M. Pramanik et al., “Automation of soil moisture sensor-based basin irrigation system,” Smart Agricultural Technology, vol. 2, art. 100032, 2022.

[4] R. Sreeram, S. A. Krishna, A. S. Kumar, S. Remya and Y. Y. Cho, “Soil moisture monitoring technologies in smart agriculture: a comprehensive review,” Farming System, vol. 4, no. 2, art. 100189, 2026.

[5] M. U. Tariq, S. M. Saqib, T. Mazhar, M. A. Khan, T. Shahzad and H. Hamam, “Edge-enabled smart agriculture framework: integrating IoT, lightweight deep learning, and agentic AI for context-aware farming,” Results in Engineering, vol. 28, art. 107342, 2025.

[6] R. K. Banoth and B. V. Ramana Murthy, “Soil image classification using transfer learning approach: MobileNetV2 with CNN,” SN Computer Science, vol. 5, no. 1, art. 199, 2024.

[7] S. Jalapur and N. Patil, “An integrated deep learning framework for soil type classification,” SN Computer Science, vol. 6, no. 3, art. 251, 2025.

[8] S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” Patterns, vol. 4, no. 9, art. 100804, 2023.

[9] M. Tan and Q. V. Le, “EfficientNetV2: smaller models and faster training,” in Proc. 38th Int. Conf. Machine Learning (ICML), PMLR vol. 139, 2021, pp. 10096–10106.

[10] D. N. K. Pandiri, R. Murugan and T. Goel, “Indian Regions Soil Image Database (IRSID),” IEEE DataPort, 2021. doi:10.21227/2zz3-f173

[11] Open-Meteo, “Free weather forecast API.” https://open-meteo.com/ (accessed Oct. 2026).